

C Programming – Operators

What is an Operator?

- An **operator** is a symbol used to perform an operation or task.
- The values on which an operator works are called **operands**.

Example:

```
a + b
```

- $+$ → Operator
- a, b → Operands

Types of Operators

C operators can be divided into:

1. Arithmetic Operators
2. Relational Operators
3. Assignment Operators
4. Logical Operators
5. Increment / Decrement Operators
6. Bitwise Operators
7. Special Operators

1. Arithmetic Operators

Used to perform **mathematical operations**.

Operator	Meaning	Example
$+$	Addition	$10 + 5 = 15$
$-$	Subtraction	$10 - 5 = 5$
$*$	Multiplication	$10 * 5 = 50$
$/$	Division	$10 / 2 = 5$
$\%$	Remainder	$10 \% 3 = 1$

Division $/$

- Division returns the **quotient**.

- If both operands are `int`, the result is also an integer.

```
10 / 2 = 5  
10 / 3 = 3
```

Important:

```
int a = 11;  
int b = 2;  
  
float c = a / b;
```

Output:

```
5.000000
```

Why?

- `a` and `b` are both `int`.
- So `a / b` performs **integer division** first.
- `11 / 2 = 5`
- Then `5` is stored in `float`.

To get decimal result:

```
float c = (float)a / b;
```

Result:

```
5.5
```

Other combinations:

```
int / float   → float  
float / int   → float  
float / float → float
```

Example:

```
12.2 / 2 = 6.1
```

Modulus %

- % gives the **remainder**.
- It is generally used with integer operands.

```
10 % 3 = 1
11 % 2 = 1
20 % 5 = 0
```

2. Relational Operators

- Relational operators are used to **compare two values**.
- They are mainly used in **conditions**.
- Result is:
 - 1 → True
 - 0 → False

Operator	Meaning	Example	Result
>	Greater than	7 > 6	1
<	Less than	5 < 6	1
>=	Greater than or equal	6 >= 6	1
<=	Less than or equal	6 <= 6	1
==	Equal to	6 == 6	1
!=	Not equal to	5 != 6	1

Examples:

```
5 > 6    // 0
7 > 6    // 1
6 > 6    // 0

5 < 6    // 1
7 < 6    // 0

5 >= 6   // 0
6 >= 6   // 1
```

```
5 <= 6    // 1
6 <= 6    // 1

5 == 6    // 0
6 == 6    // 1

5 != 6    // 1
6 != 6    // 0
```

Important Difference

```
=    → Assignment
==   → Comparison
```

Example:

```
a = 10;    // Put 10 into a

a == 10    // Check whether a is 10
```

3. Assignment Operators

Used to **assign or update values** in a variable.

Basic Assignment

```
=
```

Example:

```
int a;
a = 10;
```

Here, 10 is assigned to a.

Compound Assignment Operators

Operator	Example	Same As
=	a = 5	a = 5

Operator	Example	Same As
<code>+=</code>	<code>a += 5</code>	<code>a = a + 5</code>
<code>-=</code>	<code>a -= 5</code>	<code>a = a - 5</code>
<code>*=</code>	<code>a *= 5</code>	<code>a = a * 5</code>
<code>/=</code>	<code>a /= 5</code>	<code>a = a / 5</code>
<code>%=</code>	<code>a %= 5</code>	<code>a = a % 5</code>

Example:

```
int sum = 10;  
  
sum += 5;
```

Same as:

```
sum = sum + 5;
```

Now:

```
sum = 15
```

Bitwise Assignment Operators

```
<<=  
>>=  
&=  
^=  
|=
```

These are mainly used with **bitwise operations**.

4. Logical Operators

Used to combine or modify **conditions**.

There are three logical operators:

Operator	Name	Meaning
&&	AND	Both conditions must be true
,		OR At least one condition must be true
!	NOT	Reverses the result

AND &&

Both conditions must be true.

```
(5 < 10) && (10 > 5)
```

```
True && True = True
```

Example:

```
age >= 18 && age <= 60
```

Both conditions must be true.

OR ||

At least **one condition** must be true.

```
(5 > 10) || (10 > 5)
```

```
False || True = True
```

NOT !

Reverses the result.

```
!True → False  
!False → True
```

Example:

```
!(5 > 10)
```

Since `5 > 10` is false:

```
!False → True
```

5. Increment and Decrement Operators

Used to increase or decrease a value by **1**.

Increment **++**

```
i++;
```

Same as:

```
i = i + 1;
```

Example:

```
int i = 5;  
  
i++;
```

Now:

```
i = 6
```

Decrement **--**

```
i--;
```

Same as:

```
i = i - 1;
```

Example:

```
int i = 5;  
  
i--;
```

Now:

```
i = 4
```

Two Forms

```
i++; // Post-increment  
++i; // Pre-increment  
  
i--; // Post-decrement  
--i; // Pre-decrement
```

The difference becomes important when the operator is used inside an expression.

6. Bitwise Operators

- Bitwise operators work on the **individual bits** of integer values.
- They are commonly used in low-level programming.

Operator	Name
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

Example:

```
5 & 3
```

Binary:

```
5 = 101
3 = 011
---
001
```

Result:

```
1
```

7. Special Operators

Some operators have special purposes in C.

`sizeof`

Used to find the **size of a data type or variable in bytes**.

Example:

```
sizeof(int)
```

On many systems, this gives:

```
4
```

Example:

```
int a;

printf("%zu", sizeof(a));
```

Address Operator &

- `&` is used to get the **memory address** of a variable.
- It is commonly used with `scanf()`.

Example:

```
int age;  
  
scanf("%d", &age);
```

Here:

```
&age → address of age
```

Operators Based on Number of Operands

Operators can also be classified according to the **number of operands** they use.

1. Unary Operator

- Works with **one operand**.

Examples:

```
i++;  
i--;  
!condition;
```

Common unary operators include:

```
++  
--  
!  
~
```

Example:

```
int a = 5;  
  
a++;
```

Only one operand (**a**) is involved.

2. Binary Operator

- Works with **two operands**.

Example:

```
a + b
```

Here:

```
a → Operand  
+ → Operator  
b → Operand
```

Examples of binary operators:

```
+  
-  
*  
/  
%  
>  
<  
>=  
<=  
==  
!=  
=  
&&  
||  
&  
|  
^
```

3. Ternary Operator

- Works with **three parts/operands**.
- It is also called the **conditional operator**.
- Symbol:

```
?:
```

Syntax:

```
condition ? value_if_true : value_if_false;
```

Example:

```
int a = 10;
int b = 20;

int max = (a > b) ? a : b;
```

If:

```
a > b
```

is true → **a** is selected.

Otherwise → **b** is selected.

So:

```
max = 20
```

Quick Revision

Based on Purpose

Type	Main Operators
Arithmetic	+ - * / %
Relational	> < >= <= == !=
Assignment	= += -= *= /= %=
Logical	&& !
Increment/Decrement	++ --

Type	Main Operators
Bitwise	& ^ ~ << >>
Special	sizeof, &

Based on Number of Operands

Type	Operands	Example
Unary	1	i++
Binary	2	a + b
Ternary	3	a > b ? a : b

Tiny Complete Program

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 3;

    printf("Addition = %d\n", a + b);
    printf("Remainder = %d\n", a % b);
    printf("a is greater = %d\n", a > b);

    return 0;
}
```

Output:

```
Addition = 13
Remainder = 1
a is greater = 1
```

What is happening?

- `a + b` → Arithmetic operator
- `a % b` → Modulus operator
- `a > b` → Relational operator
- `%d` → Prints an integer
- `\n` → Moves output to the next line